Face Recognition

1. Basic Theory

Face detection algorithms can be divided into two main categories based on methods: **feature-based algorithms and image-based algorithms**.

- **Feature-based algorithms**

  Feature-based algorithms work by extracting features from images and matching them with face features. If they match, it indicates a face; otherwise, it's not. The extracted features are manually designed features, such as Haar, FHOG. After feature extraction, classifiers are used for judgment. Simply put, it uses template matching, which matches face template images with various positions in the image to be detected. The matching content is the extracted features, and then classifiers are used to determine whether there is a face.

- **Image-based algorithms**

Image-based algorithms divide the image into many small windows, then determine whether each small window contains a face. Image-based methods usually rely on statistical analysis and machine learning, finding statistical relationships between faces and non-faces through statistical analysis or learning processes for face detection. The most representative is CNN, **CNN for face detection is currently the most effective and fastest**.

- **Haar Features**

We use **machine learning** methods to complete **face detection**. First, **we need a large number of positive sample images (face images) and negative sample images (images without faces) to train the classifier**. We need to **extract features from them**. The **Haar features** shown in the figure below will be used, like our convolution kernels. **Each feature is a value**, which equals **the sum of pixel values in the black rectangle** minus **the sum of pixel values in the white rectangle**.

This tutorial uses Haar cascade classifiers, which is one of the most common and efficient object detection methods. This machine learning method trains cascade functions based on a large number of positive and negative images, then uses them to detect objects in other images. If you don't want to create your own classifier, OpenCV also includes many pre-trained classifiers that can be used for face, eye, smile detection, etc.

2. Main Code

- Start Docker

After entering the main board desktop, open the terminal and run the following command to start the corresponding Dofbot container:

```
./Docker_Ros.sh
```

Then enter the following command in the docker terminal to start jupyter-lab,

```
cd
jupyter-lab --allow-root
```

Then enter the main board's IP address plus :9999 in the browser,

```
IP:9999         // Example: 192.168.1.11:9999
```

Code path: /root/Dofbot/6.AI_Visuall/6.Face recognition.ipynb

The following code content needs to be executed step by step according to each step, and cannot be run all at once. Running the last unit will directly exit the thread.

Import Haar cascade classifier xml file

```
 self.faceDetect = cv.CascadeClassifier("haarcascade_frontalface_default.xml")
```

Face filtering

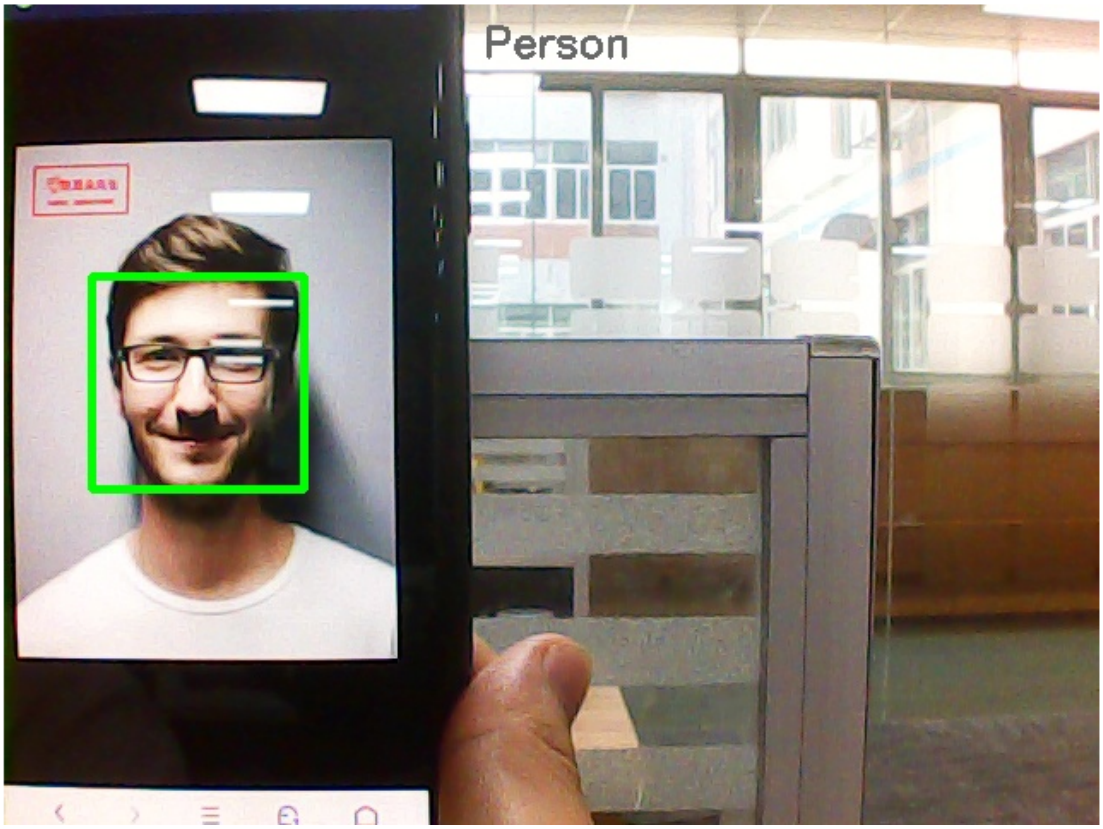
```
def face_filter(self, faces):
        '''
        Filter the face
        对人脸进行一个过滤
        '''
        if len(faces) == 0: return None
        # At present, we are looking for the face with the largest area in the
pictur
        # 目前找的是画面中面积最大的人脸
        max_face = max(faces, key=lambda face: face[2] * face[3])
        (x, y, w, h) = max_face
        # Set the minimum threshold of face detection
        # 设置人脸检测最小阈值
        if w < 10 or h < 10: return None
        return max_face
```

Recognition box selection for face

```
 def follow_function(self, img):
        img = cv.resize(img, (640, 480))
        # Copy the original image to avoid interference during processing
        # 复制原始图像,避免处理过程中干扰
        # img = img.copy()
        # Convert image to grayscale
        # 将图像转为灰度图
        gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
        # Face detection
        # 检测人脸
        faces = self.faceDetect.detectMultiScale(gray, scaleFactor=1.3,
minNeighbors=5)

        if len(faces) != 0:
            face = self.face_filter(faces)
            # Face filtering
            # 人脸过滤
            (x, y, w, h) = face
            # Draw a rectangle on the original color map
            # 在原彩图上绘制矩形
            cv.rectangle(img, (x, y), (x + w, y + h), (0, 255, 0), 4)
            cv.putText(img, 'Person', (280, 30), cv.FONT_HERSHEY_SIMPLEX, 0.8,
(105, 105, 105), 2)
```

Main thread:

```
def camera():
    global model
    # 打开摄像头 Open camera
    capture = cv.VideoCapture(0)
    while capture.isOpened():
        try:
            _, img = capture.read()
            img = cv.resize(img, (640, 480))
            img = follow.follow_function(img)
            if model == 'Exit':
                cv.destroyAllWindows()
                capture.release()
                break
            imgbox.value = cv.imencode('.jpg', img)[1].tobytes()
        except KeyboardInterrupt:capture.release()
```

After the program runs, you can see faces being detected and boxed.